东华购物 • DhuStore

在线购物系统

项 目 报 告

基于 Spring Boot + MyBatis + Vue 3 的前后端分离商城系统，
涵盖商品、购物车、订单、收货地址、用户发布商品审核与商品多图上传
等完整能力。

项目名称：DhuStore（东华购物）

技术架构：B/S · 前后端分离 · RESTful API

开发时间：2026 年 10 月

报告版本：V1.0

摘 要

本项目设计并实现了一个面向普通消费者的一站式在线购物系统 —— DhuStore（东华购物）。 系统采用前后端分离架构：后端基于 Spring Boot 4.1.1 与原生 MyBatis 构建 RESTful API， 使用 MySQL 进行数据持久化，通过 JWT 实现无状态身份认证；前端基于 Vue 3 与原生 JavaScript 模块化开发，提供商品浏览、购物车、下单结算、订单管理、个人中心与后台管理等完整交互。

在传统商城功能之上，系统进一步扩展了「用户发布商品」能力：普通用户可以提交商品发布申请， 上传 1~9 张商品图片，由管理员审核；审核通过后，系统在同一个数据库事务内完成 「更新申请状态 + 创建正式商品 + 复制图片关系」，保证数据一致性，并记录商品归属卖家。 商品图片支持阿里云 OSS 存储，未配置 OSS 时自动回退到本地磁盘，兼顾部署成本与可移植性。

后台管理模块提供数据看板、商品管理、订单管理与发布申请审核，帮助运营人员掌握销售趋势、 处理待发货订单并完成商品上架审核。系统在安全性方面做了较完整的设计，包括 BCrypt 密码加密、 JWT 鉴权、基于角色的接口权限控制、上传文件的类型与大小校验、UUID 重命名与路径穿越防护。

关键词：在线购物系统；Spring Boot；MyBatis；Vue 3；JWT；商品审核；阿里云 OSS

目 录

1 项目概述 背景、目标与角色划分

2 需求分析 功能需求与非功能需求

3 系统设计 总体架构、技术选型与模块划分

4 数据库设计 ER 模型、数据表与状态机

5 核心业务实现 下单、发布审核、多图上传与鉴权

6 系统界面展示 前端页面实拍与说明

7 接口设计 统一响应、接口清单与状态码

8 安全设计 认证授权、文件上传与数据安全

9 部署与运行 环境要求、配置与启动步骤

10 总结与展望 项目成果、不足与后续规划

1 项目概述

1.1 项目背景

随着电子商务的普及，在线购物已成为人们日常消费的重要方式。一个完整的购物系统不仅需要 商品展示与交易流程，还需要处理库存、订单状态、用户身份、支付方式以及售后等业务。 本项目在此基础上进一步引入「用户作为卖家发布商品」的 C2C 能力，使平台从单一自营商城 扩展为可以容纳第三方商品的综合性商城。

1.2 建设目标

- 构建一套结构清晰、分层明确、易于扩展的前后端分离商城系统；
- 完整实现「浏览 → 加购 → 结算 → 支付 → 发货 → 收货」的购物闭环；
- 实现用户商品发布申请与管理员审核流程，保证申请数据与正式商品数据隔离；
- 支持商品多图上传、预览、排序与详情页轮播展示；
- 提供管理员数据看板与运营工具，提升后台管理效率；
- 在安全层面落实密码加密、接口鉴权、文件上传校验等机制。

1.3 系统角色

角色	职责与权限
游客	浏览商品分类与商品列表、搜索商品、查看商品详情。无需登录。
普通用户	注册登录、管理购物车与收货地址、下单与支付、取消订单、确认收货、再次购买；提交商品发布申请并查看审核结果。
管理员	查看数据看板、维护商品信息、管理全部订单并执行发货、审核用户提交的商品发布申请。

39	10	7	2
REST 接口	数据表	业务模块	用户角色

2 需求分析

2.1 功能需求

模块	功能点	使用角色
认证与用户	注册、登录、获取当前用户、修改个人资料	全体用户
商品浏览	分类列表、商品分页、关键词搜索、搜索联想、商品详情与多图浏览	游客 / 用户
购物车	加入购物车、修改数量、勾选与全选、删除、清空选中	普通用户
收货地址	新增、编辑、删除、设置默认地址	普通用户
订单	提交订单、支付、取消、确认收货、订单列表与详情、再次购买	普通用户
商品发布	提交发布申请、上传多图、查看我的申请与审核结果	普通用户
审核管理	查看申请列表、查看详情与图片、通过或拒绝（填写原因）	管理员
后台运营	数据看板、商品增删改查、订单管理与发货	管理员
消息提醒	待发货 / 待收货角标，按页面层级切换显示位置	用户 / 管理员

2.2 非功能需求

类别	要求
安全性	密码不可逆加密存储；接口需鉴权；管理员接口做角色校验；上传文件做格式、大小、类型与路径校验。
一致性	下单扣库存、取消回滚库存、审核通过创建商品均需在事务中完成，避免脏数据。
可用性	统一响应格式与业务错误码；前端对加载、失败、空数据均有明确状态提示。
可维护性	严格分层（Controller / Service / Mapper），接口与实现分离，SQL 集中于 XML。
可移植性	图片存储支持 OSS 与本地磁盘两种模式，通过配置切换，无需修改代码。

3 系统设计

3.1 总体架构

系统采用典型的三层 B/S 架构。浏览器加载 Vue 3 单页应用，前端通过 `fetch` 调用以 `/api` 开头的 RESTful 接口；后端在进入 Controller 之前由 JWT 拦截器 完成身份解析与权限校验，随后由 Service 层组织业务与事务，Mapper 层通过 XML 执行 SQL， 最终访问 MySQL 与阿里云 OSS。

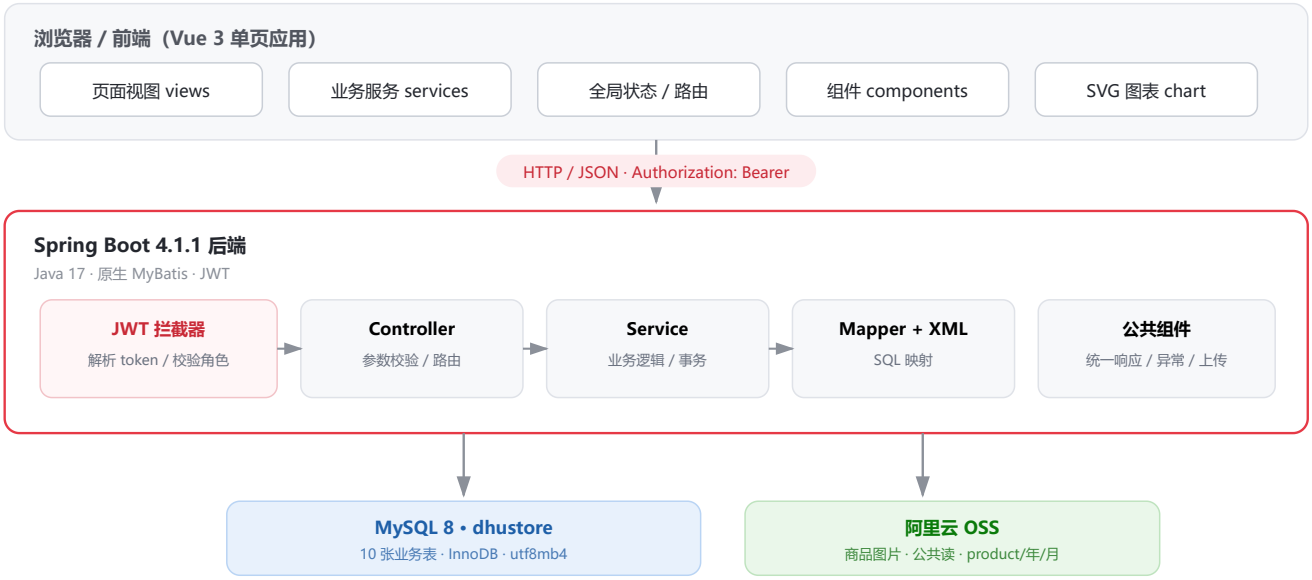


图 3-1 系统总体架构

3.2 技术选型

层次	技术	选型理由
后端框架	Spring Boot 4.1.1	自动配置、生态成熟，快速构建 REST 服务。
持久层	原生 MyBatis 3 + XML	SQL 完全可控，便于优化联表查询与批量操作。
数据库	MySQL 8	事务支持完善，适合订单与库存等强一致场景。
认证	JJWT 0.12.7	无状态 token，便于前后端分离与水平扩展。
密码加密	BCrypt（Spring Security Crypto）	自带盐值、可调强度，抗彩虹表攻击。
参数校验	Jakarta Validation	声明式校验，统一异常处理返回明确提示。
对象存储	阿里云 OSS SDK 3.17.4	稳定可靠，支持公共读与 CDN 加速。
前端	Vue 3.5 + 原生 JS 模块	轻量、无构建步骤，直接由后端托管静态资源。
图表	原生 SVG	无第三方库依赖，体积小、打印清晰。

3.3 模块划分



图 3-2 系统功能模块划分

3.4 后端分层结构

包	职责
`controller`	接收 HTTP 请求、参数校验、调用 Service 并返回统一响应。
`service`	业务编排与事务控制，是唯一允许跨 Mapper 组合操作的一层。
`mapper` + XML	数据访问接口与 SQL 映射，负责单表与联表查询。
`entity / dto`	数据库实体与请求/响应对象，隔离内外数据结构。
`common`	统一响应、业务异常、错误码、分页结果、全局异常处理。
`config`	JWT 拦截器、CORS、密码编码器、静态资源映射、示例数据初始化。
`util`	JWT 工具与当前登录用户上下文（ThreadLocal）。

4 数据库设计

4.1 实体关系

系统共设计 10 张数据表。用户与购物车、地址、订单、发布申请均为一对多关系； 商品与商品图片为一对多；申请与申请图片为一对多。审核通过时，申请图片会被复制为商品图片， 从而在数据库中形成「申请数据」与「正式商品数据」的物理隔离。

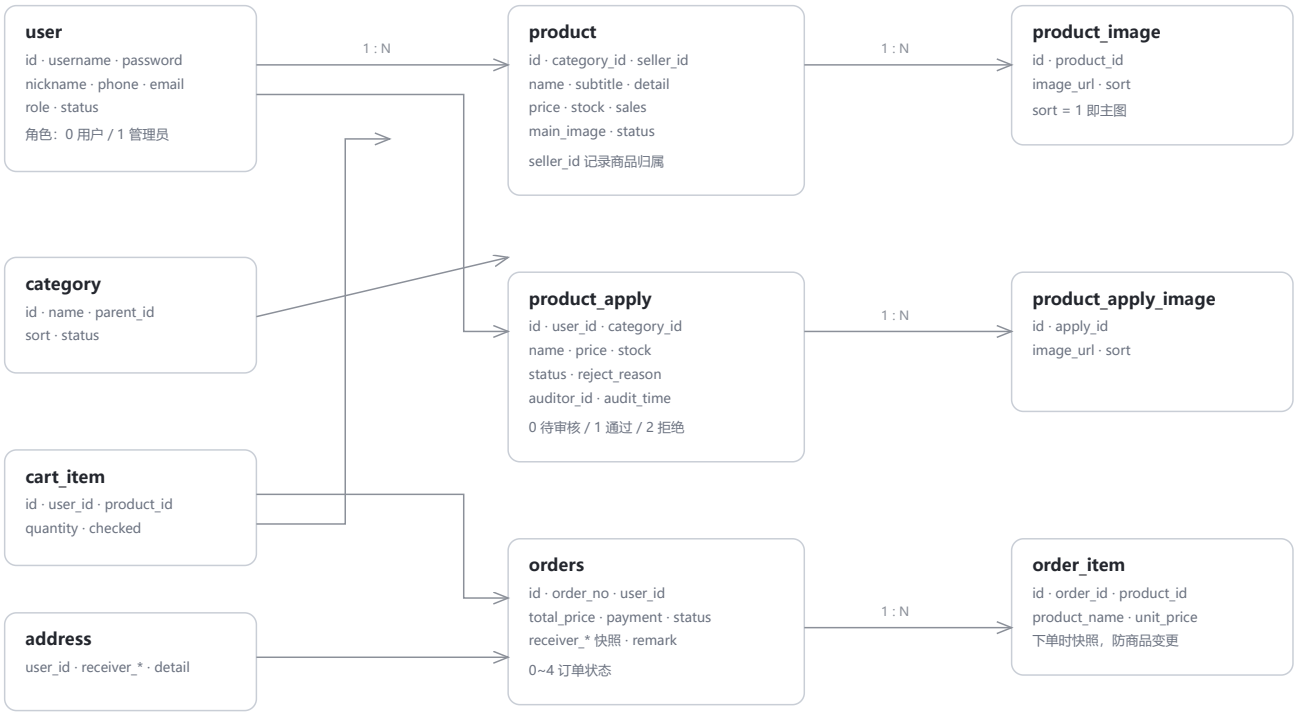


图 4-1 核心实体关系（ER）图

4.2 数据表清单

表名	说明	关键字段
`user`	用户账号与角色	username、password、role、status
`category`	商品分类（支持父子级）	name、parent_id、sort、status
`product`	正式商品	category_id、seller_id、price、stock、status
`product_image`	商品图片（1~9 张）	product_id、image_url、sort
`product_apply`	商品发布申请	user_id、status、reject_reason、auditor_id
`product_apply_image`	申请图片	apply_id、image_url、sort
`address`	收货地址	user_id、receiver_name、is_default
`cart_item`	购物车	user_id、product_id、quantity、checked
`orders`	订单主表	order_no、total_price、status、receiver_*
`order_item`	订单明细（快照）	order_id、product_name、unit_price、quantity

4.3 状态机设计

订单状态

状态	含义	触发方
0	待付款	下单后
1	已付款 / 待发货	用户支付
2	已发货 / 待收货	管理员发货
3	已完成	用户确认收货
4	已取消	用户取消（归还库存）

商品申请状态

状态	含义	说明
0	待审核	用户提交后
1	审核通过	事务内生成正式商品
2	审核拒绝	需填写拒绝原因

设计要点：订单明细保存商品名称、图片与单价快照，商品后续改价或改名不会影响历史订单； 商品归属通过 `seller_id` 记录，为后续「我的商品」「卖家中心」等功能预留扩展空间。

5 核心业务实现

5.1 购物下单流程

下单是系统中最需要保证一致性的流程。用户在购物车勾选商品后进入结算页，选择收货地址与支付方式， 提交订单时后端在同一个事务内依次完成：校验地址归属、读取勾选商品、逐项扣减库存并增加销量、 写入订单主表与明细表、清除已结算的购物车项。任一商品库存不足都会抛出业务异常并回滚整个事务。

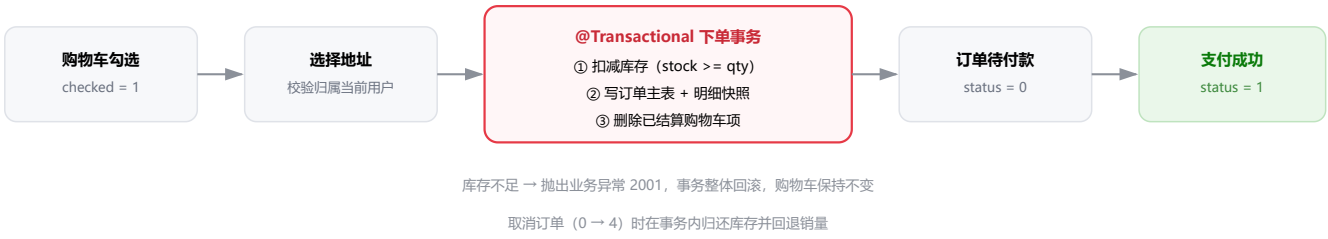


图 5-1 下单与订单状态流转

5.2 商品发布与审核流程

用户提交的商品不会直接写入正式商品表，而是先写入 `product_apply` 与 `product_apply_image`。管理员审核通过时，后端在同一个事务内完成三件事： 更新申请状态与审核信息、创建 `product` 记录（写入 `seller_id`）、 把申请图片复制到 `product_image`。这样既保证了「申请通过」与「商品创建」的原子性，也天然实现了申请数据与正式数据的隔离。

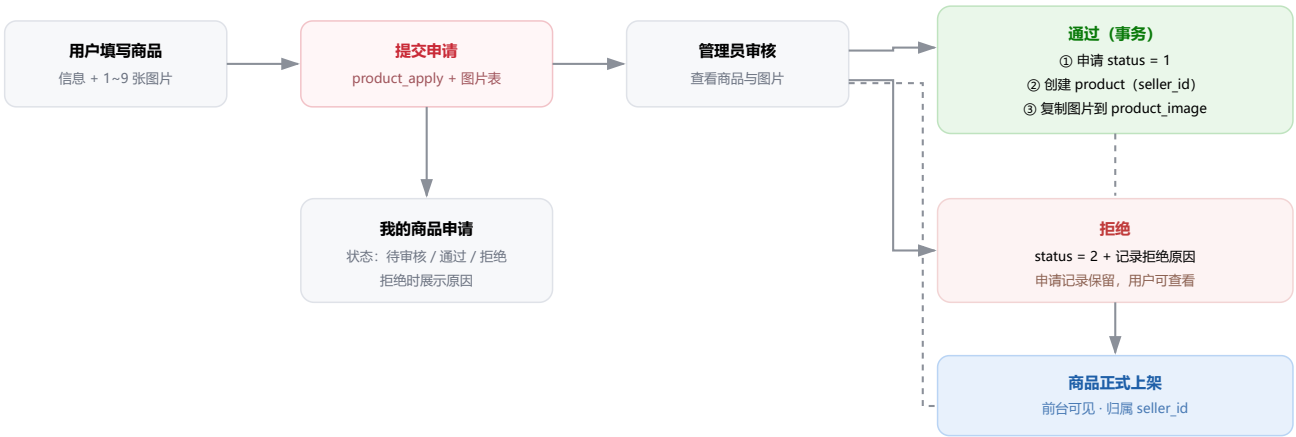


图 5-2 商品发布与审核流程

> **防重复审核：**审核前检查申请状态，只有 `status = 0` 的申请允许操作， 否则返回业务码 `2005（该申请已审核）`，避免重复生成商品。

5.3 商品多图上传

图片上传采用「前端预校验 + 后端强校验」的双重机制。前端在用户选择文件时立即校验数量、格式与大小，并调用上传接口即时预览；后端再次校验文件大小、扩展名与 Content-Type，校验通过后使用 UUID 重命名并写入 OSS，返回可访问的 URL。订单、商品详情等场景只保存 URL，不保存二进制数据。

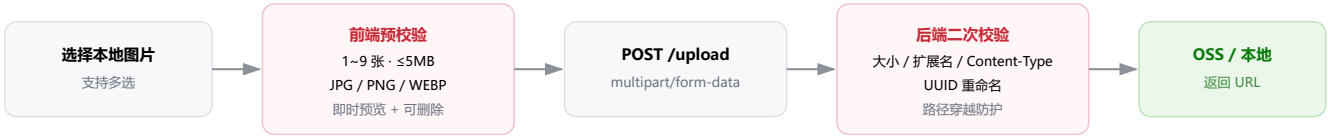


图 5-3 图片上传与校验链路

存储路径规范

```
product/{yyyy}/{MM}/{uuid}.{ext}
示例：https://dhu-stroe.oss-cn-shanghai.aliyuncs.com/product/2026/10/625c6fef11b64b91b5f32f1a9905fce6.png
```

5.4 身份认证与权限控制

用户登录成功后，后端使用 HS256 算法签发 JWT，载荷包含用户 ID、用户名与角色。前端将 token 保存在 `localStorage`，之后所有请求通过 `Authorization: Bearer <token>` 携带。后端拦截器对 `/api/**` 统一解析 token：若接口标注 `@RequireLogin` 则强制要求登录；标注 `@RequireAdmin` 时额外要求角色为 1。当前用户信息通过 `ThreadLocal` 在请求生命周期内传递，请求结束后统一清理，避免线程复用导致的数据串扰。

6 系统界面展示

以下截图均来自系统实际运行页面，数据通过后端接口实时读取。

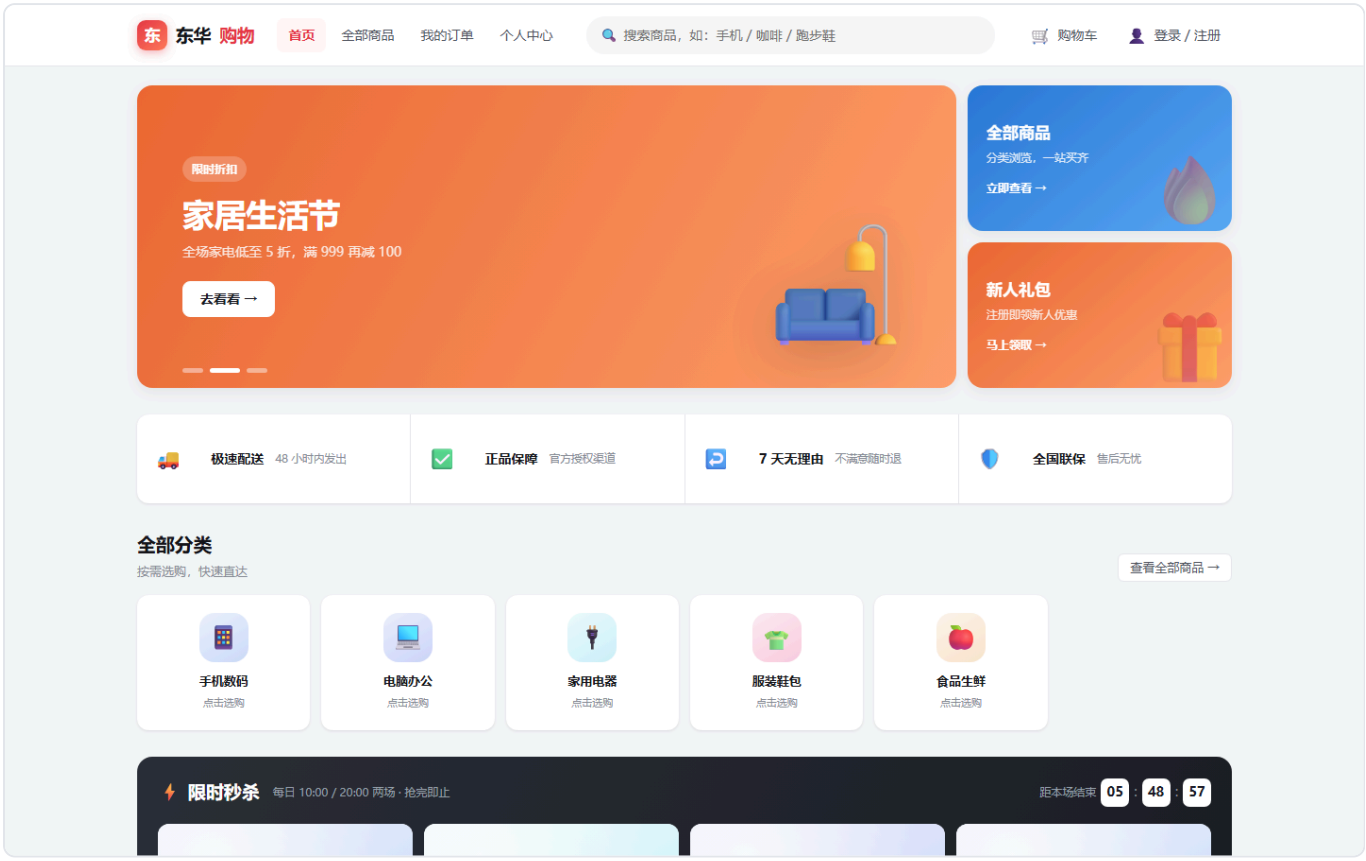


图 6-1 商城首页：轮播、服务承诺、分类入口、限时秒杀与商品推荐

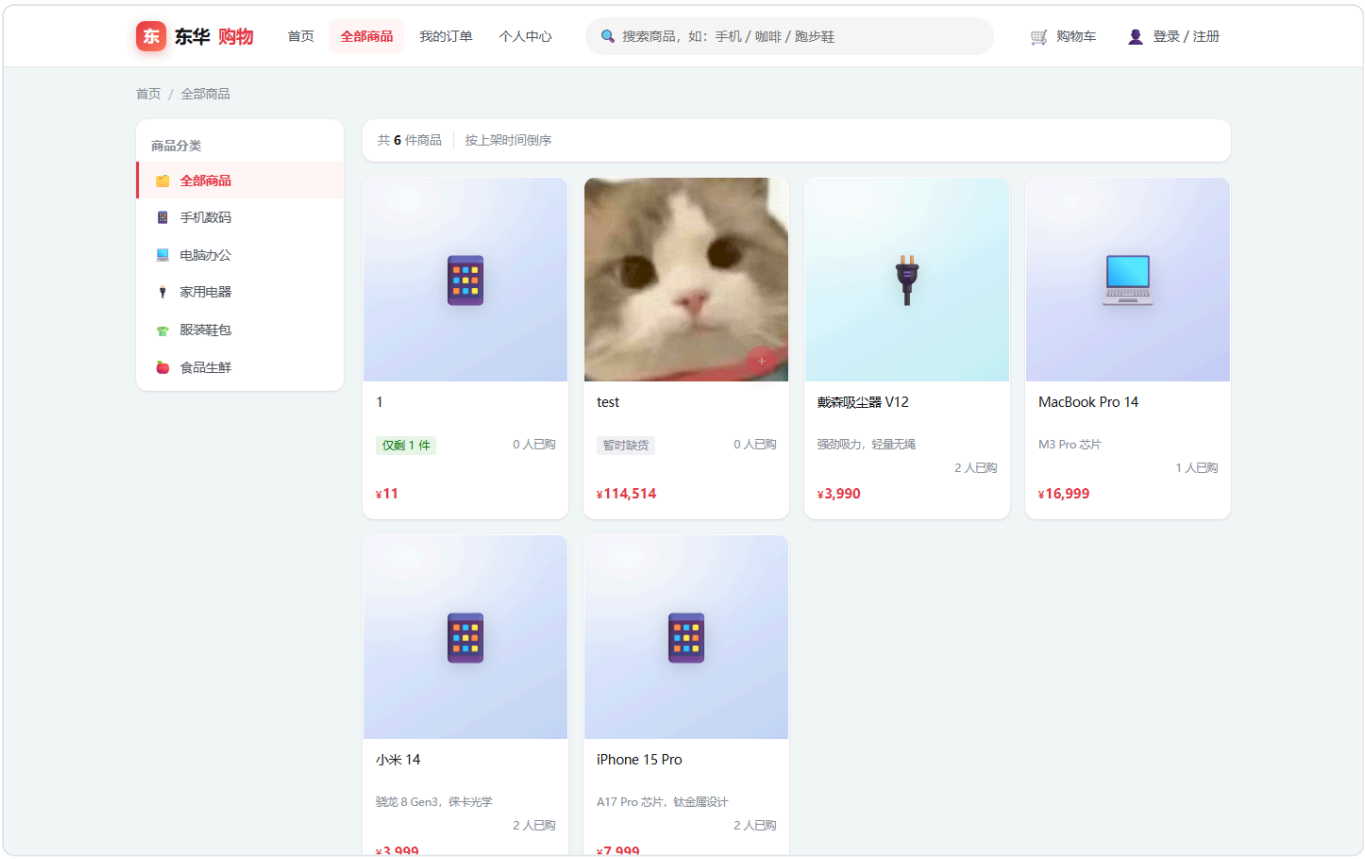


图 6-2 商品列表：左侧分类筛选、关键词搜索、分页与商品卡片

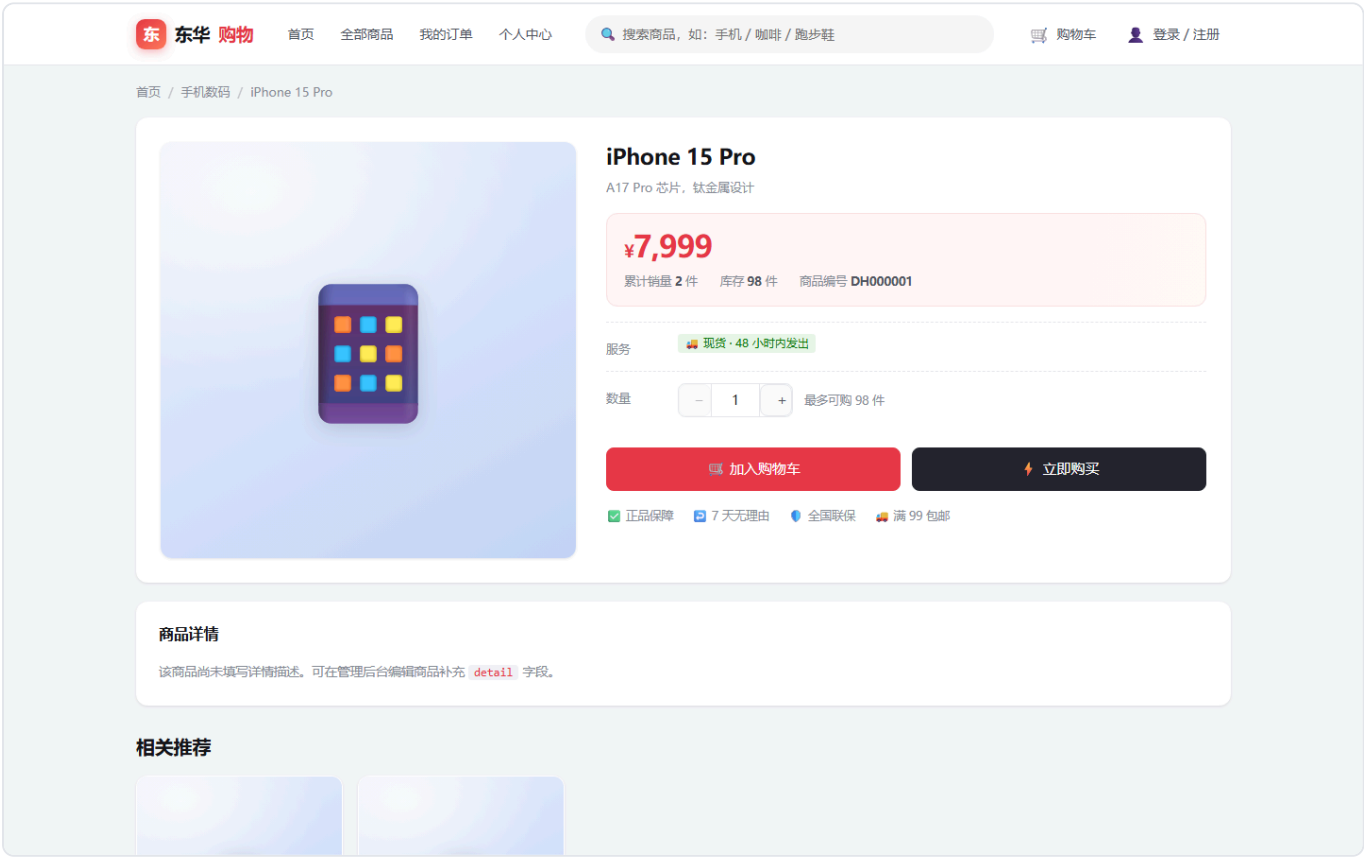


图 6-3 商品详情：主图与缩略图切换、价格库存、数量选择、加入购物车与立即购买

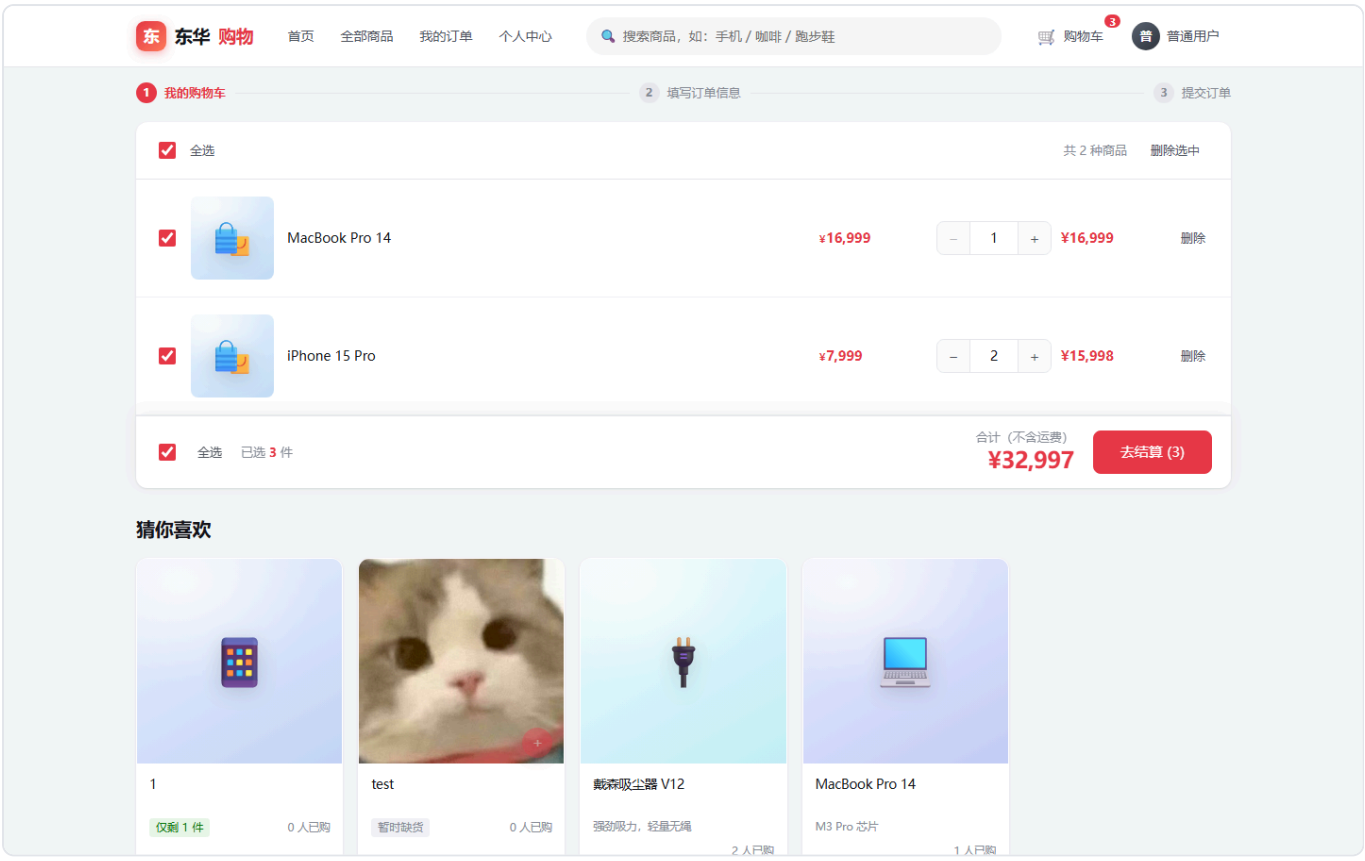


图 6-4 购物车：勾选、数量调整、小计与合计、去结算

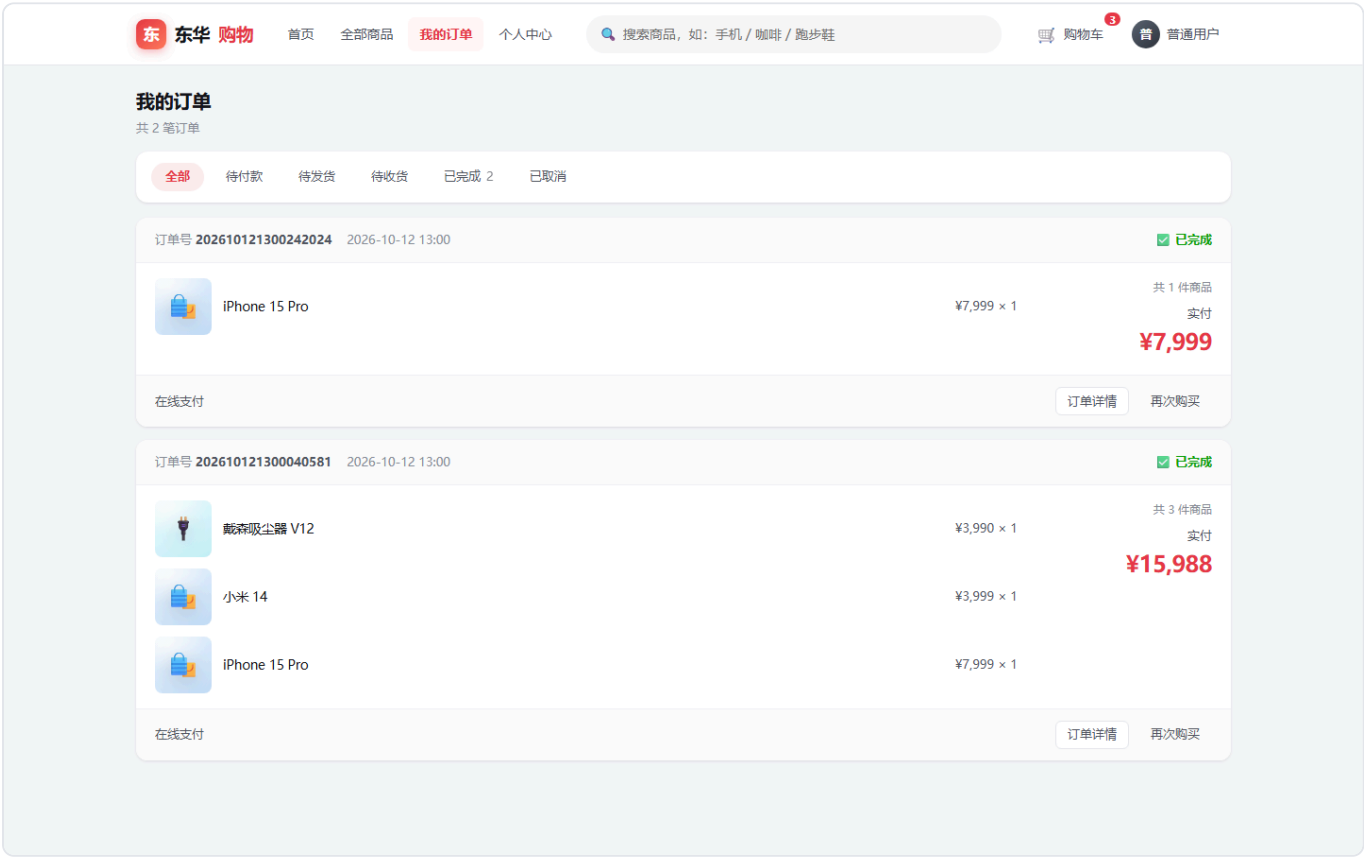


图 6-5 我的订单：状态筛选、订单详情、付款 / 取消 / 确认收货 / 再次购买

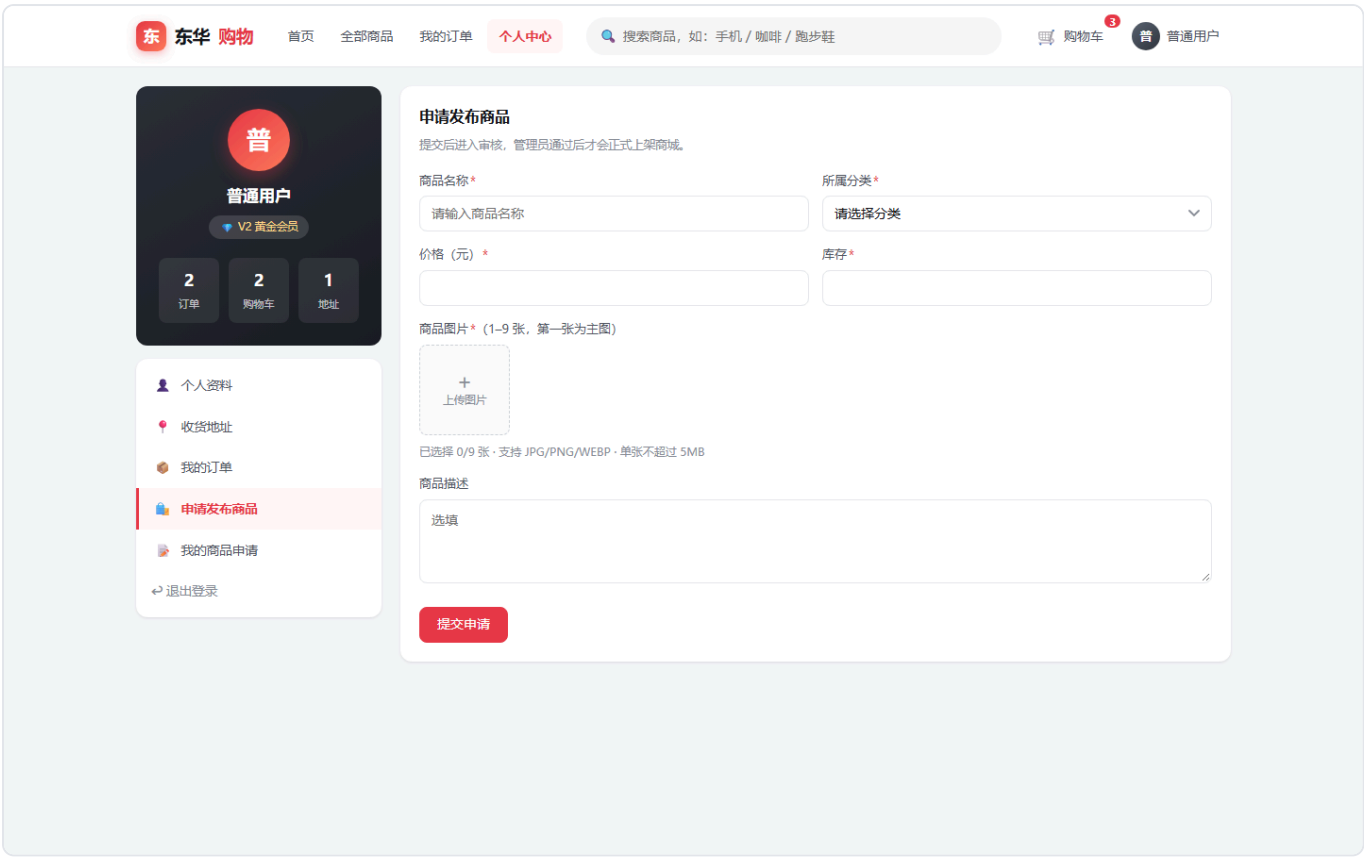


图 6-6 个人中心 · 申请发布商品：分类、价格库存、多图上传（1~9 张，首图为主图）

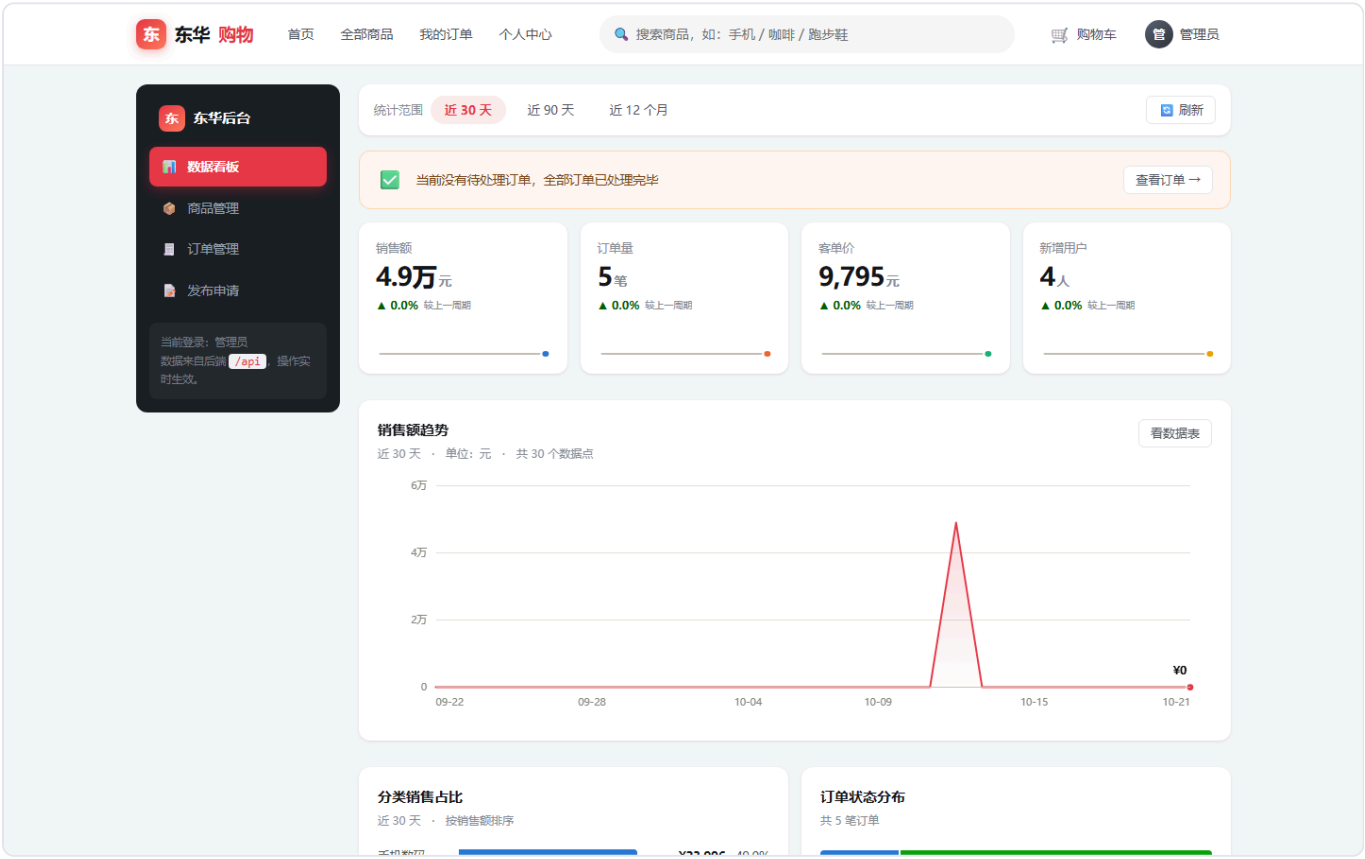


图 6-7 后台数据看板：销售额 / 订单量 / 客单价 / 新增用户 KPI 与销售趋势

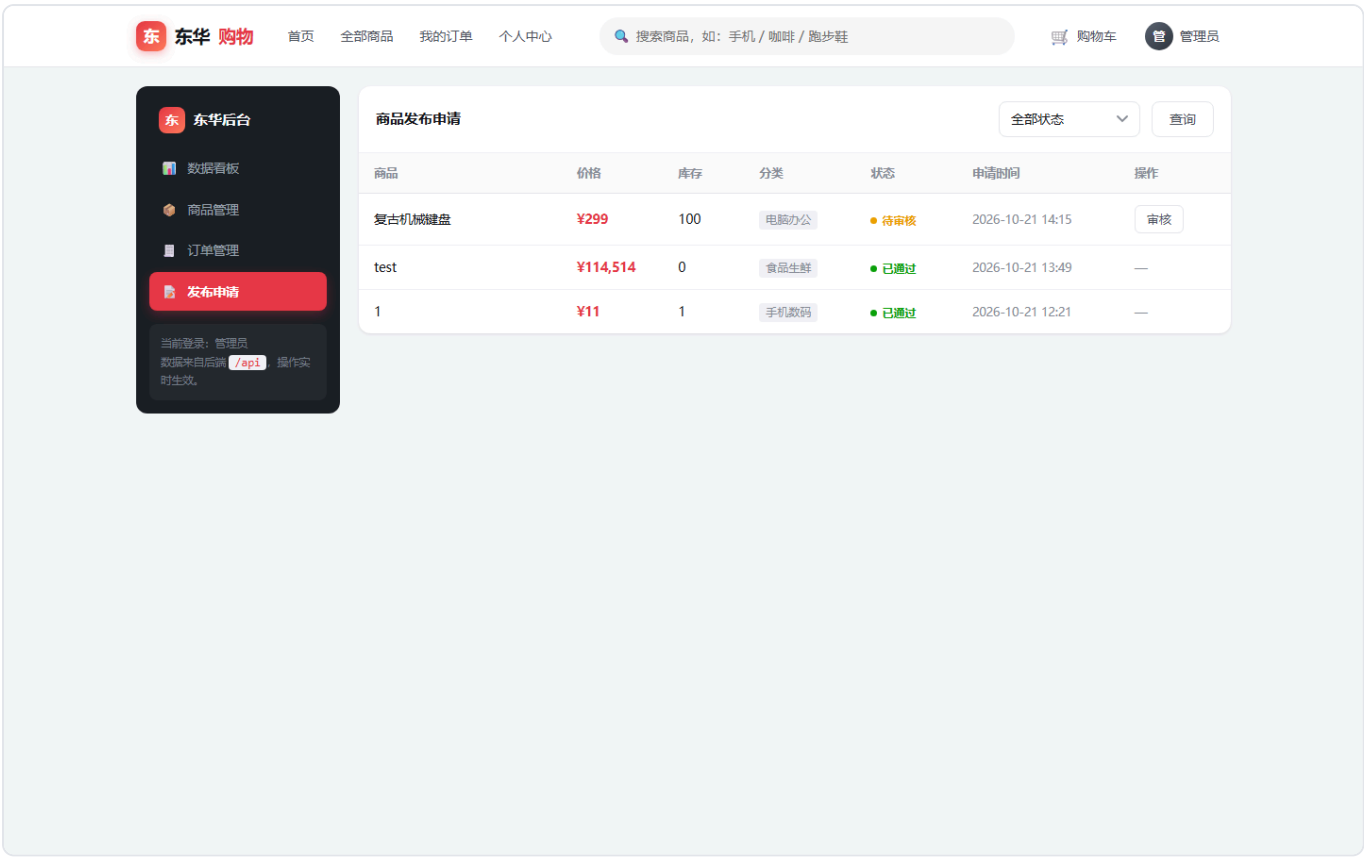


图 6-8 后台 · 商品发布申请：状态筛选、申请信息与审核入口

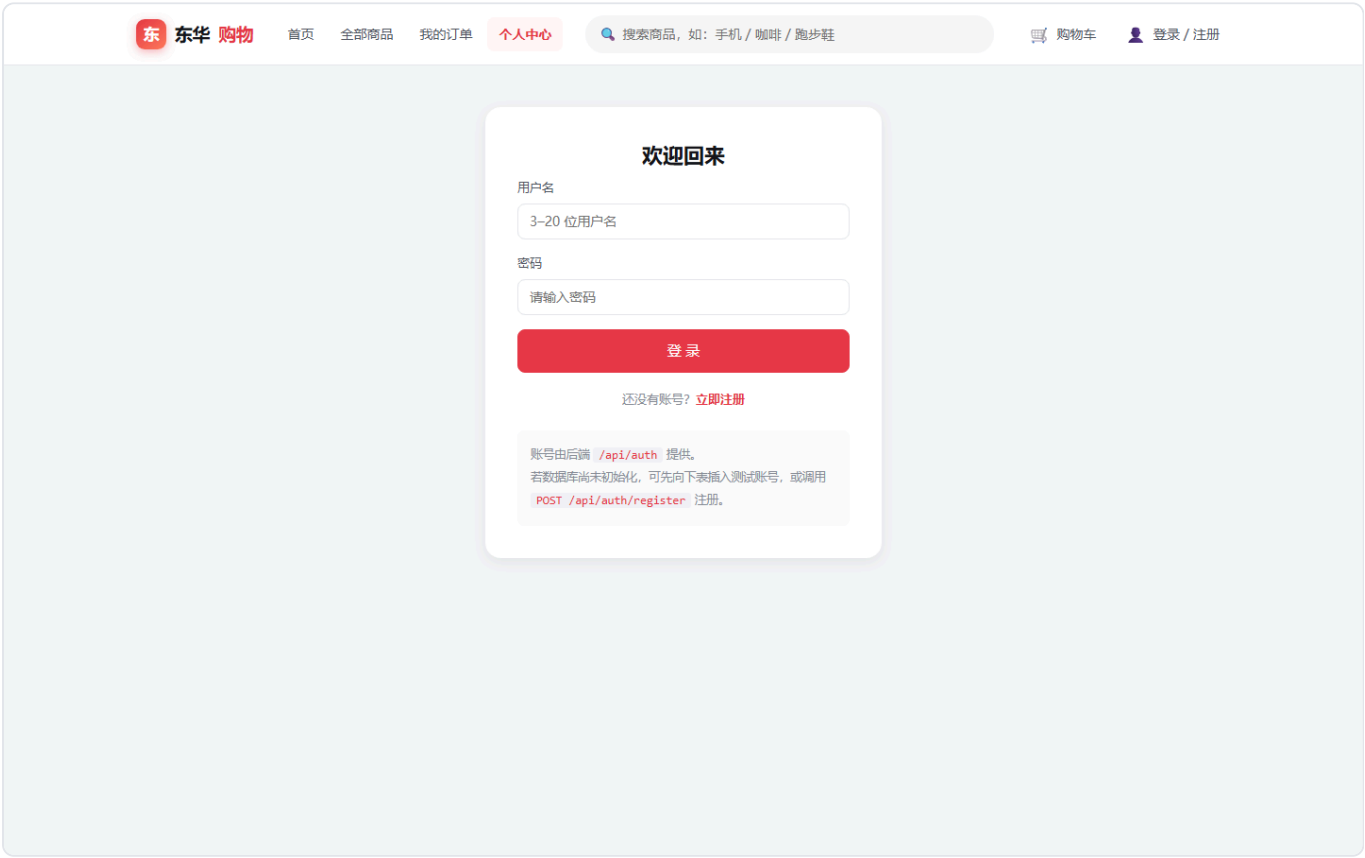


图 6-9 登录 / 注册页面

7 接口设计

7.1 统一响应

所有接口 HTTP 状态码恒为 200，业务结果通过响应体中的 `code` 区分：

```
{
  "code": 200,
  "message": "操作成功",
  "data": { ... }
}
```

前端在 `core.js` 中统一处理信封：`code = 200` 时取出 `data`；`code = 401` 时清除会话并跳转登录；其余情况抛出包含 `code` 与 `message` 的对象，由视图提示。后端由 `GlobalExceptionHandler` 保证 业务异常、参数校验异常与系统异常都返回同一结构。

7.2 接口清单（共 39 个）

方法	路径	说明	权限
POST	`/api/auth/register`	注册并返回 token	公开
POST	`/api/auth/login`	登录	公开
GET	`/api/users/me`	当前用户信息	登录
PUT	`/api/users/me`	修改个人资料	登录
GET	`/api/categories`	分类列表	公开
GET	`/api/categories/{id}`	分类详情	公开
GET	`/api/products`	商品分页（分类 / 关键词 / 状态）	公开
GET	`/api/products/{id}`	商品详情（含 images）	公开
POST	`/api/products`	新增商品	管理员
PUT	`/api/products/{id}`	修改商品	管理员
DELETE	`/api/products/{id}`	删除商品	管理员
GET	`/api/cart`	购物车列表	登录
POST	`/api/cart`	加入购物车	登录
PUT	`/api/cart/{id}`	修改数量 / 勾选	登录
PUT	`/api/cart/{id}/checked`	修改勾选状态	登录
DELETE	`/api/cart/{id}`	删除购物车项	登录
GET	`/api/addresses`	地址列表	登录
POST	`/api/addresses`	新增地址	登录
PUT	`/api/addresses/{id}`	修改地址	登录
DELETE	`/api/addresses/{id}`	删除地址	登录
PUT	`/api/addresses/{id}/default`	设为默认地址	登录
POST	`/api/orders`	提交订单	登录
GET	`/api/orders`	我的订单	登录
GET	`/api/orders/summary`	待付款 / 待收货数量	登录
GET	`/api/orders/{orderNo}`	订单详情	登录
POST	`/api/orders/{orderNo}/pay`	支付（0→1）	登录

方法	路径	说明	权限
POST	`/api/orders/{orderNo}/cancel`	取消（0→4）	登录
POST	`/api/orders/{orderNo}/confirm`	确认收货（2→3）	登录
POST	`/api/orders/{orderNo}/ship`	发货（1→2）	管理员
POST	`/api/product-apply`	提交发布申请	登录
GET	`/api/product-apply/my`	我的发布申请	登录
GET	`/api/admin/product-apply`	申请列表（可按状态）	管理员
POST	`/api/admin/product-apply/{id}/approve`	审核通过	管理员
POST	`/api/admin/product-apply/{id}/reject`	审核拒绝	管理员
POST	`/api/common/upload`	上传图片，返回 URL	登录
GET	`/api/admin/stats`	数据看板统计	管理员
GET	`/api/admin/orders`	全量订单	管理员
GET	`/api/admin/orders/pending-count`	待发货数量	管理员
GET	`/api/admin/orders/summary`	待付款 / 待发货摘要	管理员

7.3 业务状态码

code	含义	code	含义
200	操作成功	1001	用户名已存在
400	参数错误	1002	用户名或密码错误
401	未登录 / token 过期	1003	账号已被禁用
403	无权限	2001	商品库存不足
404	资源不存在	2002	购物车为空
500	服务器内部错误	2003	订单不存在
2004	订单状态不允许该操作	2005	该申请已审核
2006	单张图片超过 5MB	2007	图片格式不支持
2008	图片数量需为 1-9 张	2009	图片上传失败
2010	OSS 未配置 AccessKey	—	—

8 安全设计

8.1 认证与授权

- **密码安全**：注册时使用 BCrypt 对密码加盐哈希，数据库中不保存明文；登录时使用 `matches` 校验，接口响应中不返回密码字段。
- **无状态认证**：JWT 使用 HS256 签名，密钥长度不少于 32 字节，token 默认 24 小时过期。
- **接口级权限**：`@RequireLogin` 与 `@RequireAdmin` 由拦截器统一执行， 不依赖前端隐藏按钮，普通用户无法通过直接调用接口越权。
- **上下文隔离**：用户信息存放在 `ThreadLocal`，请求结束即清理，防止线程复用串号。

8.2 文件上传安全

- 限制单文件 5MB、单请求 50MB（Spring Multipart 配置 + 业务校验双重限制）；
- 白名单校验扩展名（jpg / jpeg / png / webp）与 Content-Type，不信任前端 `accept`；
- 文件名完全由服务端生成（UUID），避免恶意文件名与覆盖攻击；
- 本地存储时校验目标路径必须位于上传根目录内，防止路径穿越；
- 上传接口要求登录，未登录请求返回 401。

8.3 业务与数据安全

- **越权防护**：地址、购物车、订单的查询与修改均附带 `user_id` 条件， 用户只能操作自己的数据；订单查询还会校验归属。
- **库存并发**：扣减库存使用 `WHERE stock >= #{quantity}` 的条件更新， 通过影响行数判断是否成功，避免超卖。
- **事务一致性**：下单、取消订单、审核通过均使用 `@Transactional`。
- **XSS 防护**：商品详情按纯文本渲染（不使用 `v-html`），避免后台录入内容注入脚本。
- **敏感配置**：数据库密码、JWT 密钥与 OSS AccessKey 存放于被 gitignore 的本地配置文件， 仓库中仅保留环境变量占位符。

9 部署与运行

9.1 环境要求

组件	版本要求	说明
JDK	17 及以上	Spring Boot 4 要求 Java 17+
Maven	3.6+	构建与启动
MySQL	8.0+	字符集 utf8mb4
浏览器	Chrome / Edge 最新版	支持 ES6 与 Fetch API
阿里云 OSS	可选	未配置时自动使用本地磁盘

9.2 数据库初始化

```
# 全新数据库
mysql -uroot -p < src/main/resources/db/schema.sql
mysql -uroot -p < src/main/resources/db/data.sql

# 已有旧库：按顺序执行增量迁移
mysql -uroot -p < src/main/resources/db/migration_1_product_apply.sql
mysql -uroot -p < src/main/resources/db/migration_2_product_image.sql
```

9.3 配置项

配置项	说明
`spring.datasource.*`	数据库连接，敏感值通过环境变量或本地配置注入
`jwt.secret`	JWT 签名密钥，至少 32 字节
`storage.type`	`local` 或 `oss`
`aliyun.oss.*`	endpoint、bucket、AccessKey 等 OSS 参数
`file.upload-dir`	本地存储目录，默认 `uploads`
`spring.servlet.multipart.*`	单文件与整请求大小限制

9.4 启动与访问

```
mvn spring-boot:run
# 访问 http://localhost:8080
```

应用启动时会幂等创建两个示例账号，便于验收：

账号	密码	角色
`admin`	`123456`	管理员（role = 1）
`user`	`123456`	普通用户（role = 0）

10 总结与展望

10.1 项目成果

- 完成了一个可运行、可演示的前后端分离在线购物系统，覆盖商品、购物车、订单、地址等核心交易链路；
- 实现了用户发布商品与管理员审核的 C2C 扩展，并通过事务与状态校验保证申请与正式商品的数据一致性；
- 实现了商品多图上传、预览与详情页展示，接入阿里云 OSS 并支持本地存储回退；
- 完成了管理员数据看板、商品管理、订单管理与发布申请审核等运营能力；
- 形成了统一响应、统一异常、统一鉴权的后端基础设施，后续新增模块成本较低。

10.2 存在的不足

- 消息提醒采用登录与操作后的拉取刷新，尚未实现 WebSocket 实时推送；
- 商品图片排序按上传顺序，暂未支持拖拽调整；
- 商品与申请列表使用 `LIKE` 模糊查询，数据量增大后需要引入全文检索；
- 下单成功后的「支付」为模拟状态流转，尚未接入真实支付渠道；
- 缺少自动化测试与持续集成流程。

10.3 后续规划

1. **卖家中心：**基于 `seller_id` 提供「我的商品」「销售统计」「上下架」能力；
2. **实时通知：**使用 WebSocket 或 SSE 推送新订单、发货、审核结果；
3. **交易增强：**订单超时自动取消、退款流程、优惠券、运费模板；
4. **性能优化：**引入 Redis 缓存热点商品与验证码，图片接入 CDN；
5. **工程质量：**补充单元测试与接口测试，接入 CI 自动构建与部署；
6. **运维能力：**接入日志聚合、监控告警与健康检查。

> **结语：**本项目从需求分析、数据库设计到接口实现与前端交互，完整走通了一个电商系统的核心闭环， 并在用户发布商品、审核事务、多图存储等环节进行了较深入的设计实践。系统结构清晰、层次分明， 为后续扩展为真正的多卖家商城奠定了基础。